

Illimité

Illimité is a premium, state-of-the-art e-commerce platform curated for modern home styling, objects, and furnishings. It features a complete end-to-end shopping journey with a modern frontend architecture and a scalable Firebase-backed backend.

1. Technology Stack

Frontend Architecture

- **Framework:** Angular v20
- **State & Logic:** Reactive design with Angular Signals, `computed()` values, and RxJS streams.
- **UI Components:** Angular Material v20 (Toolbar, Menu, Buttons, etc.) and custom components.
- **Styling:** Premium vanilla CSS tailored for a minimalist, modern aesthetic (supporting slide carousels, responsive grid layouts, and glassmorphism micro-animations).
- **Authentication & Database Sync:** Firebase Client SDK (@angular/fire).

Backend Architecture

- **Runtime & Framework:** Node.js with Express v5.x
- **SDK Integrations:** Firebase Admin SDK (for Firestore, Authentication, and Storage management).
- **File Uploads:** Multer with Firebase Storage bucket integration.
- **Utilities:** `dotenv` (environment configuration), `cors`, `uuid`.

Database & Storage

- **Database:** Google Cloud Firestore (NoSQL Document Store)
- **Asset Storage:** Google Cloud Storage for product image hosting

2. Directory Structure

```
illimite-git/
├── illimite-client/           # Angular 20 Frontend Application
│   ├── src/
│   │   ├── app/
│   │   │   ├── components/  # Page components (Login, Cart, Admin, etc.)
│   │   │   ├── core/       # Guards, Interceptors, Shared Models, and Services
│   │   │   ├── services/    # Product API data services
│   │   │   ├── app.ts       # Main App Component class (logic, catalog filter/search)
│   │   │   └── app.html     # Main App template and Shell layout
│   │   └── package.json
│   └── illimite-server/      # Express & Firebase Admin Backend
│       ├── config/          # Firebase Admin initialization & Service keys
│       ├── middleware/      # Firebase Auth Token Verification
│       ├── routes/          # Modular Router files (upload, cart)
│       ├── services/        # Backend business logic (cart services)
│       ├── server.js         # Express application entry point & core API endpoints
│       ├── seed.js          # Firestore database seeder utility
│       └── package.json
├── docs/                    # Technical documentation and Postman collections
├── react-pages/              # Legacy/reference React components (for migration review)
├── firestore.rules           # Security rules for Cloud Firestore
└── storage.rules             # Security rules for Cloud Storage
```

3. Data Models & Database Schema

The Firestore database relies on root-level collections designed for rapid queries:

1. Categories (`categories/{id}`)

Stores curated collections of products.

- **id:** Unique string identifier (e.g., `kitchen`, `lamps`, `home-decor`)
- **name:** Display name (e.g., `"Home Decor"`)

- **slug**: URL-friendly string identifier
- **imageUrl**: Cover image URL

2. Products (products/{slug})

Product items using stable slugs as the Firestore Document ID to prevent duplicates.

- **id / slug**: Unique product slug (e.g., aura-pendant-light)
- **name**: Product title
- **description**: HTML-stripped product detail text
- **price**: Current purchase price (integer, INR)
- **originalPrice**: Price before discount
- **discountPercent**: Calculated discount percentage
- **stock**: Real-time available inventory quantity
- **categoryId / categoryName**: Category relationship attributes
- **imageUrl**: Primary display image
- **imageUrls**: Array of strings for product image galleries

3. Carts (carts/{uid})

Active user shopping carts (synced for logged-in users).

- **uid**: Firebase Authentication User ID
- **items**: Array of objects (productId, qty, price, etc.)
- **updatedAt**: ISO timestamp

4. Orders (orders/{id})

Processed order records for checkout and admin fulfillment workflows.

- **id**: Automatically generated Firestore ID
- **userId**: Customer's Firebase User ID
- **items**: Array of ordered product items with purchase price details
- **total**: Invoice total value
- **status**: Current fulfillment state (pending | paid | processing | shipped | delivered | cancelled)

4. Setup Instructions

Please refer to the detailed environment setup instructions in [SETUP_INSTRUCTIONS.md \(/SETUP_INSTRUCTIONS.md\)](#).

For API contracts and data models, check the `docs/` directory.

5. Frontend Navigation & Router Map

The customer is guided through the app via protected guards (authGuard and adminGuard):

- **Public Routes**: / (Catalog Home), /login, /register, /products/:slug
- **Protected Routes (Requires Auth)**: /profile, /cart, /wishlist, /checkout, /orders, /orders/:id, /orders/:id/confirmation
- **Admin Routes (Requires Admin Role)**: /admin (Admin Dashboard)

6. Project Status

The project is currently at **75-80% completion**, representing a functional MVP. It features complete end-to-end user journeys from product browsing and adding to cart, to user authentication, secure checkout, and basic admin management views.